

Spis treści

1	Stabilność sortów	2
2	Bubble sort	2
2.1	Działanie	2
2.2	Złożoność	2
2.3	Stabilność	2
3	Insertion sort	2
3.1	Działanie	2
3.2	Zastosowanie	2
3.3	Złożoność	3
3.4	Stabilność	3
4	Selection sort	3
4.1	Działanie	3
4.2	Złożoność	3
4.3	Stabilność	3
5	Merge sort	4
5.1	Działanie	4
5.2	Zastosowanie	4
5.3	Złożoność	4
5.4	Stabilność	4
6	Heap sort	4
6.1	Działanie	4
6.2	Zastosowanie	4
6.3	Złożoność	4
6.4	Stabilność	4
7	Quick sort	5
7.1	Działanie	5
7.2	Zastosowanie	5
7.3	Złożoność	5
7.4	Stabilność	5
8	Counting sort	5
8.1	Działanie	5
8.2	Zastosowanie	5
8.3	Złożoność	5
8.4	Stabilność	6
9	Radix sort	6
9.1	Działanie	6
9.2	Zastosowanie	6
9.3	Złożoność	6
9.4	Stabilność	6

10 Bucket sort	6
10.1 Działanie	6
10.2 Zastosowanie	6
10.3 Złożoność	6
10.4 Stabilność	7

1 Stabilność sortów

Stabilny sort nie psuje poprzedniego układu. Jak masz listę graczy już posortowaną alfabetycznie. Teraz chcesz ich posortować drugi raz, ale po zdobytych punktach. Jeśli algorytm jest **stabilny**, to gracze z taką samą liczbą punktów dalej będą ułożeni alfabetycznie.

Totalny gamechanger i must-have, gdy musisz posortować dane po kilku kryteriach po kolei.

Sorty proste $O(n^2)$

raczej brak realnych zastosowań

2 Bubble sort

2.1 Działanie

Lecisz parami od lewej do prawej. Jak element po lewej jest większy od tego po prawej, to robisz swap. I tak w kółko, aż największe elementy "wypłyną" na sam koniec tablicy jak bąbelki.

2.2 Złożoność

Czasowa: $O(n^2)$

Pamięciowa: $O(1)$

2.3 Stabilność

Stabilny

3 Insertion sort

3.1 Działanie

Wyobraź sobie, że układasz karty w ręce. Bierzesz kolejną z kupki i wpychasz ją w odpowiednie miejsce wśród tych, które już masz ułożone w dłoni.

3.2 Zastosowanie

Fajnie się sprawdza, jak tablica jest już prawie posortowana albo jest totalnie malutka. Często używany wewnątrz Quick Sorta dla małych podtablic.

3.3 Złożoność

Czasowa: $O(n^2)$ (ale optymistyczna dla prawie posortowanych to $O(n)$)

Pamięciowa: $O(1)$

3.4 Stabilność

Stabilny.

4 Selection sort

4.1 Działanie

Skanujesz całą tablicę, znajdujesz absolutnie najmniejszy element i wrzucasz go na sam początek (zamieniasz z pierwszym). Potem szukasz najmniejszego z pozostałych i dajesz na drugie miejsce. I tak do oporu.

4.2 Złożoność

Czasowa: $O(n^2)$

Pamięciowa: $O(1)$

4.3 Stabilność

Niestabilny

Sorty szybkie $O(n \log n)$

z porównaniami

5 Merge sort

5.1 Działanie

Klasyczne podejście Dziel i Zwyciężaj (Divide and Conquer). Tniemy tablicę w połowie, potem te połówki znów na pół, aż zostaną nam z tego pojedyncze elementy. Potem zaczynamy je ze sobą sklejać, ale układając je od razu rosnąco.

5.2 Zastosowanie

Top do sortowania linked list, jedyny sort szybki $O(n \log n)$ który jest stabilny.

5.3 Złożoność

Czasowa: $O(n \log n)$ (realnie najwolniejszy szybki przez dużą stałą)

Pamięciowa: $O(n)$

5.4 Stabilność

Stabilny.

6 Heap sort

6.1 Działanie

Budujemy z naszej tablicy strukturę zwaną kopcem (max-heap) – to takie drzewo, gdzie stary jest zawsze większy od dzieci. Potem zdejmujemy korzeń (bo wiemy, że jest największy z całej paczki), wrzucamy go na koniec tablicy i naprawiamy to, co zostało z kopca. Powtarzamy, aż posortujemy całość.

6.2 Zastosowanie

Potrzebujesz 100% pewności $O(n \log n)$? Jakiś krytyczny system coś? To jest to

6.3 Złożoność

Czasowa: $O(n \log n)$

Pamięciowa: $O(1)$

6.4 Stabilność

Niestabilny.

7 Quick sort

7.1 Działanie

Wybierasz sobie jeden element (tzw. pivot). Potem rzucasz wszystkie mniejsze od pivota na lewo od niego, a większe na prawo. W ten sposób pivot łąduje na swoim ostatecznym miejscu. Następnie robisz dokładnie to samo (rekurencyjnie) dla lewej i prawej strony.

7.2 Zastosowanie

Defaultowy sort ogólnego przeznaczenia. Kiedy nie wiesz, jak dokładnie wyglądają dane, zależy ci na prędkości średniej i

7.3 Złożoność

Czasowa średnia: $O(n \log n)$

Czasowa najgorsza: $O(n^2)$

Pamięciowa: $O(\log n)$

7.4 Stabilność

Niestabilny

Sorty szybkie $O(n)$

bez porównań

8 Counting sort

8.1 Działanie

Robimy sobie nową tablicę z licznikami. Przelatujemy przez dane i po prostu zliczamy: "mam trzy piątki, dwa zera". Potem na podstawie tych liczników odtwarzamy po kolei posortowaną tablicę.

8.2 Zastosowanie

Masz 10^8 liczb z wąskiego zakresu np. 0-1000? To jest to

8.3 Złożoność

Czasowa: $O(n + k)$

Pamięciowa: $O(k)$

gdzie k to zakres liczb

8.4 Stabilność

Stabilny (jak się go mądrze napisze od końca).

9 Radix sort

9.1 Działanie

Sortujemy liczby (lub stringi) cyfra po cyfrze. Najpierw układamy wszystko np. po jednościach (używając jakiegoś stabilnego sortu, najczęściej Counting Sorta), potem po dziesiątkach, setkach itd. Na sam koniec całe liczby wyjdą nam ładnie poukładane.

9.2 Zastosowanie

Masz 10 milionów kodów długości 12 do posortowania? To jest to

9.3 Złożoność

Czasowa: $O(d \cdot (n + k))$

Pamięciowa: $O(n + k)$

gdzie k to zakres liczb, a d to największa liczba (ilość cyfr)

9.4 Stabilność

Stabilny (wręcz MUSI taki być, inaczej cały algorytm sypie się na starcie – dlatego wewnętrznie bazuje na innym stabilnym сорcie).

10 Bucket sort

10.1 Działanie

Rozrzucamy nasze dane do tzw. "wiader" (np. liczby 0-9 do pierwszego, 10-19 do drugiego). Skoro wiemy, że dane są w miarę równo rozłożone, to w wiaderkach wyląduje mało elementów. Sortujemy sobie każde wiaderko osobno (najczęściej Insertion sortem) i zlewamy wszystko z powrotem w jedną całość.

10.2 Zastosowanie

Dane są równomiernie rozłożone? W zadaniu jest wspomniany **rozkład jednostajny**? To jest to

10.3 Złożoność

Czasowa średnia: $O(n + k)$

Czasowa najgorsza: $O(n^2)$ - gdy wszystkie liczby wpadną do jednego wiadra

Pamięciowa: $O(n + k)$

gdzie k to liczba wiader, typowa liczba wiader to n

10.4 Stabilność

Stabilny (o ile sort, którego używasz wewnątrz wiaderek, np. Insertion sort, jest stabilny).